

自定义按键

1.硬件接线

本案例使用的是MSPM0机器人扩展板，里面所使用到的外设不需要自己额外接线，只需要将亚博MSPM0G3507核心板插入到扩展板中。

2.代码解释

- empty.c

```
int main(void)
{
    USART_Init();

    //清除定时器中断标志 Clear timer interrupt flag
    NVIC_ClearPendingIRQ(TIMER_0_INST_INT_IRQN);
    //使能定时器中断 Enable Timer Interrupt
    NVIC_EnableIRQ(TIMER_0_INST_INT_IRQN);
    //定时器开始计时 Timer start
    DL_TimerA_startCounter(TIMER_0_INST);

    while (1)
    {
        //按键检测设置在<timer.c>中
        //keystroke detection settings in <timer.c>
    }
}
```

主函数中执行了系统的初始化与定时器的开启。本工程将按键检测放置在了<timer.c>文件中。

- timer.c

```
void TIMER_0_INST_IRQHandler(void)
{
    switch( DL_TimerG_getPendingInterrupt(TIMER_0_INST) )
    {
        case DL_TIMER_IIDX_ZERO://如果是0溢出中断 If it is a 0 overflow interrupt
            Key_Handle();//按键触发检测 Key Trigger Detection
            systick_counter++; // 每1ms自动+1 +1 per sencond
            break;

        default:
            break;
    }
}

uint32_t Get_Time(void)
{
}
```

```

    return systick_counter;
}

```

- TIMER_0_INST_IRQHandler:1ms的定时器中断函数，用于持续检测按键的输入，同时用于计数
- Get_Time:用于获取总计数

- key.c

```

// 定义按键句柄 - Define key handle
Key_t key1 = {
    .GPIOx = KEY_PORT,
    .GPIO_Pin = KEY_K1_PIN,
    .state = KEY_STATE_RELEASED,
    .presTime = 0,
    .debounceTime = 0
};

/* 消抖时间（单位：ms） - Debounce time (ms) */
#define DEBOUNCE_DELAY 20 // 典型值：10-50ms / Typical value: 10-50ms

/**
 * @brief 按键扫描函数（非阻塞式） - Key scan function (non-blocking)
 * @param key 按键句柄指针 / Pointer to KeyHandle
 * @param currentTime 当前系统时间（单位：ms） / Current system time (ms)
 * @param longPressThreshold 长按时间阈值（单位：ms） / Long press threshold (ms)
 * @return KeyEvent 返回按键事件 / Returns key event
 */
KeyEvent Key_Scan(Key_t* key, uint32_t currentTime, uint32_t longPressThreshold)
{
    // 读取按键电平：0表示按下，1表示释放 / Read pin level: 0=pressed, 1=released
    uint8_t isPressed = 0;

    if(DL_GPIO_readPins(key->GPIOx, key->GPIO_Pin) == 0)
    {
        isPressed = 1;
    }

    switch (key->state) {
        /* 状态1: 按键释放 - State 1: Key released */
        case KEY_STATE_RELEASED:
            if (isPressed) {
                key->state = KEY_STATE_DEBOUNCE; // 进入消抖状态 / Enter
                debounce state
                key->debounceTime = currentTime; // 记录消抖开始时间 / Record
                debounce start time
            }
            break;

        /* 状态2: 消抖检测 - State 2: Debounce checking */
        case KEY_STATE_DEBOUNCE:
            // 消抖时间到达后检测稳定状态 / Check stable state after debounce delay
            if (currentTime - key->debounceTime >= DEBOUNCE_DELAY) {

```

```

        if (isPressed) {
            key->state = KEY_STATE_PRESSED;    // 确认按下 / Confirm press
            key->presTime = currentTime;        // 记录按下时间 / Record
press time
        } else {
            key->state = KEY_STATE_RELEASED;    // 抖动误触发 / False
trigger due to bounce
        }
    }
    break;

    /* 状态3: 已按下 - State 3: Pressed */
    case KEY_STATE_PRESSED:
        if (!isPressed) {
            key->state = KEY_STATE_RELEASED;    // 释放触发短按 / Release
triggers short press
            return KEY_EVENT_SHORT;            // 返回短按事件 / Return
short press event
        } else if (currentTime - key->presTime >= longPressThreshold) {
            key->state = KEY_STATE_LONG;        // 触发长按 / Trigger long
press
            return KEY_EVENT_LONG;            // 返回长按事件 / Return
long press event
        }
        break;

    /* 状态4: 长按已触发 - State 4: Long press triggered */
    case KEY_STATE_LONG:
        if (!isPressed) {
            key->state = KEY_STATE_RELEASED;    // 释放后重置状态 / Reset
state after release
        }
        break;
    }

    return KEY_EVENT_NONE;    // 默认无事件 / Default: no event
}

void Key_Handle(void)
{
    int16_t LongPressThreshold = 700; // 按键扫描时长按的阈值: 500ms    Threshold for
long press during key scanning: 500ms
    static uint32_t lastTick = 0;    // 初始时间    initial time
    uint32_t currentTick = Get_Time();
    static int task_flag = 1;

    // 每10ms检测一次按键 - Check key every 10ms
    if (currentTick - lastTick >= 10) {
        lastTick = currentTick;

        KeyEvent event = Key_Scan(&key1, currentTick, LongPressThreshold);

        switch (event) {
            case KEY_EVENT_SHORT:
                // 处理短按 Handle short press

```

```
//          printf("short press\r\n");
LED_Toggle();
break;
case KEY_EVENT_LONG:
    // 处理长按 Handle long press

    break;
default:
    break;
    }
}
}
```

- Key_t key1:创建一个按键结构体，用于存储配置按键的引脚以及按下时长。
- Key_Scan:通过获取定时器的计数，来判断按键按下的时间，用于消抖与判断长按还是短按。
- Key_Handle:按键检测处理函数，可以更改按键长按的阈值，让长接触发的更快或者更慢。在switch判断事件中，也可以根据短按或者长按，触发不同的功能。这里将短按事件的处理为反转LED灯的电平。

- led.c

```
void LED_Toggle(void)
{
    DL_GPIO_togglePins(LED_PORT, LED_D1_PIN);
}
```

- LED_Toggle:LED电平反转，如果灯是灭的，按下按键会反转为亮。

3.程序现象

烧入程序后，再按下NRST键进行复位，就可以按下扩展板上的K1按键来点亮或者熄灭核心板上的D1丝印的LED灯了。

